



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Integrantes:

Ochoa Aviles Edgar

Carrera:

Ingeniería En Sistemas Computacionales

Materia:

Tópicos De IA

Tarea:

Solución(es) a una función matemática usando el algoritmo PSO **BONUS(1)**

Profesor:

Zuriel Dathan Mora Felix

Grupo:

12:00 - 13:00 PM

Fecha:

4/11/2025

GITHUB LINK:

[https://github.com/edgarochoa00/Topicos-De-Inteligencia-Artificial/tree/main/MODULO%203/TAREA%20BONUS%201%20Soluci%C3%B3n\(es\)%20a%20una%20funci%C3%B3n%20matem%C3%A1tica%20usando%20el%20algoritmo%20PSO](https://github.com/edgarochoa00/Topicos-De-Inteligencia-Artificial/tree/main/MODULO%203/TAREA%20BONUS%201%20Soluci%C3%B3n(es)%20a%20una%20funci%C3%B3n%20matem%C3%A1tica%20usando%20el%20algoritmo%20PSO)

1. Introducción

El presente informe desarrolla la implementación y comparación de dos algoritmos metaheurísticos ampliamente utilizados en la optimización numérica: la Optimización por Enjambre de Partículas (PSO, por sus siglas en inglés) y la Evolución Diferencial (DE). Ambos pertenecen a la familia de métodos estocásticos poblacionales que buscan soluciones óptimas aproximadas en espacios de búsqueda complejos y multimodales.

El objetivo de este trabajo es aplicar ambos algoritmos sobre funciones matemáticas de referencia (benchmarks), observar su comportamiento, generar visualizaciones de convergencia y realizar un análisis comparativo entre sus rendimientos.

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization.
Storn, R., & Price, K. (1997). Differential Evolution

2. Optimización por Enjambre de Partículas (PSO)

2.1. Fundamentos teóricos

El algoritmo PSO se basa en la observación del comportamiento colectivo de grupos animales, como bandadas de aves o bancos de peces. Cada partícula representa una solución candidata dentro del espacio de búsqueda y actualiza su posición y velocidad en función de dos factores:

- Su **mejor posición personal** alcanzada hasta el momento (*pbest*).
- La **mejor posición global** encontrada por todo el enjambre (*gbest*).

Las ecuaciones de actualización son:

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t))$$

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

donde w es el factor de inercia, c_1 y c_2 son los coeficientes cognitivo y social respectivamente, y r_1, r_2 son números aleatorios uniformes entre 0 y 1.

Citas:

Eberhart, R. C., & Shi, Y. (2001). *Particle swarm optimization: developments, applications and resources*. Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546).

Clerc, M., & Kennedy, J. (2002). *The particle swarm - explosion, stability, and convergence in a multidimensional complex space*. IEEE Transactions on Evolutionary Computation, 6(1), 58–73.

2.2. Implementación

La implementación desarrollada se encuentra en el archivo `pso.py`, donde se definen dos clases principales:

- **Particula:** representa cada individuo del enjambre, con su posición, velocidad y mejor valor encontrado.
- **OptimizadorPSO:** gestiona el conjunto de partículas, actualiza sus posiciones y evalúa el desempeño global.

El programa inicializa las partículas con posiciones y velocidades aleatorias dentro de los límites especificados y aplica el algoritmo PSO de tipo *global-best*. Se emplean límites de búsqueda, amortiguamiento de velocidad en colisiones y decaimiento lineal del factor de inercia.

Poli, R., Kennedy, J., & Blackwell, T. (2007).

2.3. Resultados

El algoritmo se probó con distintas funciones matemáticas de referencia: **Esfera**, **Rosenbrock**, **Rastrigin**, **Ackley** y **Griewank**.

Ejemplo de parámetros utilizados:

Parámetro	Valor
Número de partículas	40
Iteraciones máximas	200
$w_{iniciow_inicio}$	0.9
w_{finw_fin}	0.4
c_1, c_2	1.7

El PSO mostró una rápida convergencia hacia el mínimo global en funciones suaves como *Esfera* y *Ackley*, con valores finales cercanos a 10^{-6} después de unas 50 iteraciones.

Figura 1. Convergencia del PSO

(Insertar aquí la imagen generada: *Esfera_PSO_convergencia.png*)

Figura 2. Posiciones finales de las partículas

(Insertar: *Esfera_PSO_posiciones.png*)

Bansal, J. C., Sharma, H., Jadon, S. S., & Clerc, M. (2011).

3. Evolución Diferencial (DE)

3.1. Fundamentos teóricos

La Evolución Diferencial es un algoritmo evolutivo propuesto por Storn y Price (1997) que utiliza operaciones genéticas adaptadas para espacios continuos. Cada individuo en la población se transforma mediante una **mutación diferencial**, seguida de un **cruce binomial** y una **selección competitiva**.

La estrategia implementada en este trabajo es **rand/1/bin**, definida como:

$$v_i = x_{r1} + F \cdot (x_{r2} - x_{r3})$$
$$v_{i,j} = \begin{cases} v_{i,j} & \text{si } \text{rand}_j \leq CR \\ x_{i,j} & \text{en otro caso} \end{cases}$$
$$u_{i,j} = \begin{cases} v_{i,j} & \text{si } \text{rand}_j \leq CR \\ x_{i,j} & \text{en otro caso} \end{cases}$$
donde F es el factor de mutación y CR la tasa de cruce.

Citas:

Storn, R., & Price, K. (1997). *Differential Evolution*

3.2. Implementación

El código fuente se encuentra en el archivo de .py. Se define la clase `EvolucionDiferencial`, que realiza los pasos de:

1. Inicialización de la población dentro de los límites de búsqueda.
2. Aplicación de mutación diferencial rand/1.
3. Cruce binomial entre el vector objetivo y el vector mutante.
4. Selección uno-a-uno reemplazando los individuos peores por mejores.
5. Registro del mejor valor encontrado por iteración.

Ejemplo de parámetros:

Parámetro	Valor
Tamaño de población	60
Iteraciones máximas	200
F	0.8
CR	0.9

Das, S., & Suganthan, P. N. (2011).

3.3. Resultados

La DE demostró una convergencia más gradual que PSO, pero con una mayor capacidad para escapar de mínimos locales. En la función *Rastrigin*, alcanzó valores mínimos en el orden de 10^{-4} después de aproximadamente 150 iteraciones.

Figura 3. Convergencia del DE
(Insertar: *Rastrigin_DE_convergencia.png*)

Figura 4. Posiciones finales de la población
(Insertar: *Rastrigin_DE_posiciones.png*)

Qin, A. K., Huang, V. L., & Suganthan, P. N. (2009).

4. Comparación entre PSO y DE

Aspecto	PSO	DE
Inspiración	Comportamiento social de enjambres	Evolución biológica diferencial
Parámetros clave	w, c_1, c_2	F, CR
Velocidad de convergencia	Alta	Media
Diversidad poblacional	Tiende a reducirse rápidamente	Se mantiene más tiempo
Escapatoria de mínimos locales	Media	Alta
Facilidad de implementación	Alta	Media
Adecuado para	Problemas suaves y continuos	Problemas multimodales complejos

En general, PSO converge más rápido, pero puede estancarse en mínimos locales debido a la pérdida de diversidad. DE, por otro lado, mantiene una exploración más amplia del espacio de búsqueda y obtiene resultados más estables en funciones con múltiples óptimos locales.

Das, S., Abraham, A., & Konar, A. (2008).

5. Conclusiones

1. Ambos algoritmos logran aproximar el óptimo global en funciones de prueba clásicas, pero con dinámicas de búsqueda distintas.
2. **PSO** destaca por su velocidad de convergencia en problemas unimodales, mientras que **DE** muestra mejor rendimiento en espacios multimodales o con ruido.
3. La implementación modular permitió la comparación directa entre ambos métodos, así como la generación de visualizaciones y reportes reproducibles.
4. En términos de código, ambos algoritmos se ejecutaron sin errores y produjeron resultados consistentes con la teoría.

Citas:

Engelbrecht, A. P. (2012). *Computational Intelligence*:

6. Referencias

- Kennedy, J., & Eberhart, R. (1995). *Particle swarm optimization*. Proceedings of ICNN'95.
- Storn, R., & Price, K. (1997). *Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces*. *Journal of Global Optimization*, 11(4), 341–359.
- Clerc, M., & Kennedy, J. (2002). *The particle swarm - explosion, stability, and convergence in a multidimensional complex space*. *IEEE Transactions on Evolutionary Computation*, 6(1), 58–73.
- Das, S., & Suganthan, P. N. (2011). *Differential Evolution: A Survey of the State-of-the-Art*. *IEEE Transactions on Evolutionary Computation*, 15(1), 4–31.
- Poli, R., Kennedy, J., & Blackwell, T. (2007). *Particle swarm optimization – An overview*. *Swarm Intelligence*, 1(1), 33–57.
- Engelbrecht, A. P. (2012). *Computational Intelligence: An Introduction*. Wiley.